

Chuyên đề: Các Thuật toán Xử lý Đoạn & Khoảng trên Đường thẳng (1D Interval Processing)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

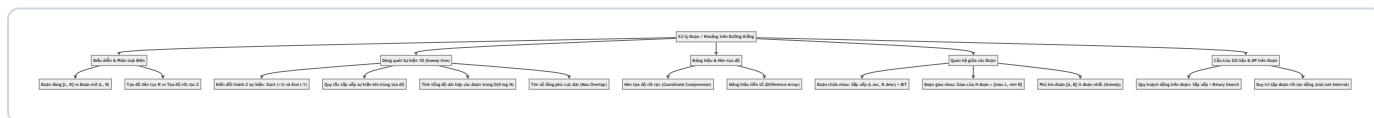
Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong Lập trình thi đấu (Competitive Programming), các bài toán liên quan đến **đoạn thẳng / khoảng thời gian (Intervals / Segments)** trên trục số thực $\mathbb{R}$ hoặc trục số nguyên $\mathbb{Z}$ là lớp bài toán xuất hiện với tần suất cực kỳ cao. Mỗi đoạn thẳng thường được định nghĩa bởi một cặp điểm biên $[L_i, R_i]$ với $L_i \leq R_i$.

Các thao tác cốt lõi trên tập hợp các đoạn thẳng bao gồm:

- Gộp các đoạn giao nhau (Merge Overlapping Intervals).
- Tính tổng độ dài hợp các đoạn (Length of Union of Intervals).
- Tìm số tầng phủ cực đại (Maximum Stacking / Overlap Count).
- Phủ đoạn thẳng mục tiêu bằng số đoạn ít nhất (Minimum Interval Cover).
- Đếm quan hệ hình học giữa các đoạn (Đoạn chứa nhau, đoạn giao nhau, đoạn rời nhau).
- Chọn tập đoạn không giao nhau có tổng trọng số lớn nhất (Weighted Interval Scheduling).



2. Phân loại Biên & Biểu diễn Toán học

2.1. Phân loại Đoạn thẳng

Một đoạn thẳng trên trục tọa độ có thể ở một trong các dạng sau:

- Đoạn đóng (Closed Interval):** $[L, R] = \{x \in \mathbb{R} \mid L \leq x \leq R\}$. Hai đoạn $[1, 3]$ và $[3, 5]$ có giao nhau tại điểm $x = 3$.
- Đoạn mở (Open Interval):** $(L, R) = \{x \in \mathbb{R} \mid L < x < R\}$. Hai đoạn $(1, 3)$ và $(3, 5)$ không giao nhau.
- Đoạn nửa mở (Half-open Interval):** $[L, R) = \{x \in \mathbb{R} \mid L \leq x < R\}$. Đây là chuẩn thường được dùng trong Computer Science (ví dụ: `vector.begin()` và `vector.end()`). Hai đoạn $[1, 3)$ và $[3, 5)$ tiếp xúc nhưng không chồng lấn.

2.2. Độ dài Đoạn thẳng trên Trục Số

- Trên trục số thực $\mathbb{R}$:

Độ dài hình học của đoạn $[L, R]$ là:

$$\text{Length} = R - L$$

- Trên trục số nguyên $\mathbb{Z}$ (Đếm số điểm nguyên thuộc đoạn):

Số lượng điểm nguyên thuộc đoạn đóng $[L, R]$ là:

$$\text{Count} = R - L + 1$$

★ Quan trọng:

Cần đọc kỹ đề bài xem yêu cầu tính **độ dài hình học** ($\mu = R - L$) hay tính **số lượng điểm nguyên rời rạc** ($R - L + 1$).

⚡ 3. Các Mô hình Xử lý Đoạn Kinh điển

3.1. Hợp các Đoạn Giao nhau (Merge Intervals)

Bài toán: Cho N đoạn thẳng $[L_i, R_i]$. Hãy gộp tất cả các đoạn có phần giao nhau thành các đoạn rời rạc phân biệt.

Thuật toán $O(N \log N)$:

1. Sắp xếp các đoạn theo đầu mút trái L_i tăng dần. Nếu $L_i = L_j$, sắp xếp theo R_i tăng dần.
2. Khởi tạo đoạn hiện tại: $[cur_L, cur_R] = [L_0, R_0]$.
3. Duyệt lần lượt qua các đoạn tiếp theo $[L_i, R_i]$:
4. Nếu $L_i \leq cur_R$: Đoạn thứ i giao hoặc tiếp xúc với đoạn hiện tại $\implies$ gộp vào bằng cách cập nhật:

$$cur_R = \max(cur_R, R_i)$$

5. Nếu $L_i > cur_R$: Đoạn thứ i hoàn toàn rời rạc với đoạn hiện tại $\implies$ lưu $[cur_L, cur_R]$ vào kết quả, và bắt đầu đoạn mới $[cur_L, cur_R] = [L_i, R_i]$.
6. Lưu đoạn cuối cùng vào danh sách kết quả.

```

struct Interval {
    long long l, r;
};

vector<Interval> mergeIntervals(vector<Interval>& intervals) {
    if (intervals.empty()) return {};
    sort(intervals.begin(), intervals.end(), [](const Interval& a, const Interval& b) {
        if (a.l != b.l) return a.l < b.l;
        return a.r < b.r;
    });

    vector<Interval> merged;
    long long cur_l = intervals[0].l;
    long long cur_r = intervals[0].r;

    for (size_t i = 1; i < intervals.size(); ++i) {
        if (intervals[i].l <= cur_r) {
            cur_r = max(cur_r, intervals[i].r);
        } else {
            merged.push_back({cur_l, cur_r});
            cur_l = intervals[i].l;
            cur_r = intervals[i].r;
        }
    }
    merged.push_back({cur_l, cur_r});
    return merged;
}

```

- **Tổng độ dài hợp các đoạn:** Sau khi gộp thành các đoạn rời rạc $[L'_j, R'_j]$, tổng độ dài hợp chỉ đơn giản là:

$$\text{Total Length} = \sum_j (R'_j - L'_j)$$

3.2. Dòng quét Sự kiện 1D (1D Sweep-line / Event Points)

Kỹ thuật Dòng quét (Sweep-line) xem mỗi đoạn thẳng $[L_i, R_i]$ như hai sự kiện rời rạc xảy ra theo dòng thời gian từ trái sang phải:

- Sự kiện **BẮT ĐẦU (START)** tại $x = L_i$: Thêm một lớp phủ mới (+1).
- Sự kiện **KẾT THÚC (END)** tại $x = R_i$: Mất đi một lớp phủ (-1).

Quy tắc Ưu tiên Sắp xếp khi Trùng Tọa độ:

Nếu có hai sự kiện có cùng tọa độ x , thứ tự xử lý quyết định tính đúng đắn của thuật toán:

- **Với đoạn đóng** $[L, R]$: Hai đoạn $[1, 3]$ và $[3, 5]$ được xem là giao nhau tại điểm 3. Do đó, tại $x = 3$, sự kiện START của đoạn sau **phải được xử lý TRƯỚC** sự kiện END của đoạn trước.
- **Với đoạn mở** (L, R) hoặc **nửa mở** $[L, R)$: Hai đoạn $[1, 3)$ và $[3, 5)$ không giao nhau tại 3. Do đó, sự kiện END của đoạn trước **phải được xử lý TRƯỚC** sự kiện START của đoạn sau.

Trực quan hóa Dòng quét Sự kiện (Sweep-line):

Đoạn 1: [L1 = 1 ----- R1 = 5]

Đoạn 2: [L2 = 3 ----- R2 = 8]

Đoạn 3: [L3 = 6 ---- R3 = 9]

Trục x:	1	3	5	6	8	9
Sự kiện:	+1	+1	-1	+1	-1	-1
Số tầng:	1	2	1	2	1	0

Thuật toán tìm số tầng phủ cực đại (Max Overlap):

```
int maxOverlap(const vector<Interval>& intervals) {
    // Sự kiện: {tọa độ x, loại sự kiện}
    // Đối với đoạn đóng: START (+1) có độ ưu tiên cao hơn END (-1), gán type: START = +1, END = -1
    vector<pair<long long, int>> events;
    for (const auto& iv : intervals) {
        events.push_back({iv.l, +1}); // Bắt đầu
        events.push_back({iv.r, -1}); // Kết thúc
    }

    // Sắp xếp: Tọa độ x tăng dần. Nếu x bằng nhau, sự kiện +1 đứng trước -1
    sort(events.begin(), events.end(), [](const pair<long long, int>& a, const pair<long long, int>& b) {
        if (a.first != b.first) return a.first < b.first;
        return a.second > b.second; // +1 đứng trước -1
    });

    int cur_layers = 0;
    int max_layers = 0;
    for (const auto& ev : events) {
        cur_layers += ev.second;
        max_layers = max(max_layers, cur_layers);
    }
    return max_layers;
}
```

3.3. Bài toán Phủ Đoạn Tối thiểu (Minimum Interval Cover)

Bài toán: Cho tập N đoạn thẳng $[L_i, R_i]$ và một đoạn mục tiêu $[A, B]$. Hãy chọn ra số lượng đoạn thẳng **ít nhất** sao cho hợp của chúng phủ kín hoàn toàn $[A, B]$.

Chiến lược Tham lam:

- Sắp xếp các đoạn theo L_i tăng dần.
- Duy trì mốc đã phủ được: `cur_end = A`.
- Trong các đoạn có $L_i \leq \text{cur_end}$, ta luôn tham lam chọn đoạn có R_i **lớn nhất** vươn xa nhất về bên phải.
- Cập nhật `cur_end = max_R`. Nếu tại bất kỳ bước nào mà không tìm được đoạn nào vươn xa hơn `cur_end`, kết luận **không thể phủ kín** (trả về -1).

5. Dừng lại khi $\text{cur_end} \geq B$.

6. **Độ phức tạp:** $O(N \log N)$ (chỉ tốn bước sắp xếp ban đầu, vòng lặp tham lam chạy trong $O(N)$ bằng hai con trỏ).

3.4. Đếm Quan hệ giữa các Đoạn: Đoạn Chứa nhau (Nested Ranges)

Bài toán (CSES 2168 & 2169): Cho N đoạn thẳng $[L_i, R_i]$. Với mỗi đoạn i :

- Kiểm tra xem đoạn i có chứa ít nhất một đoạn khác hay không ($[L_j, R_j] \subseteq [L_i, R_i]$).
- Kiểm tra xem đoạn i có bị đoạn nào khác chứa nó hay không ($[L_i, R_i] \subseteq [L_j, R_j]$).

Kỹ thuật Sắp xếp Đa khóa + Cây Fenwick (BIT):

Để kiểm tra đoạn j nằm trong đoạn i , điều kiện là:

$$L_i \leq L_j \quad \text{và} \quad R_j \leq R_i$$

- **Quy tắc sắp xếp đặc biệt:**

Sắp xếp các đoạn theo:

- L tăng dần.
- Nếu L bằng nhau, sắp xếp theo R giảm dần.

Khi sắp xếp như vậy, với mọi $j > i$: ta chắc chắn có $L_j \geq L_i$.

Điều kiện đoạn j nằm trong đoạn i chỉ còn phụ thuộc vào duy nhất một chiều:

$$R_j \leq R_i$$

1. **Đếm xem đoạn i có bị đoạn nào trước đó chứa không:**

Duyệt từ trái sang phải ($i = 0 \rightarrow N - 1$). Một đoạn trước đó chứa đoạn i khi và chỉ khi nó có $R \geq R_i$.

Ta chỉ cần duy trì giá trị lớn nhất của R đã gặp (max_R). Nếu $\text{max_R} \geq R_i \implies$ đoạn i bị chứa!

2. **Đếm xem đoạn i có chứa đoạn nào phía sau không:**

Duyệt từ phải sang trái ($i = N - 1 \rightarrow 0$). Một đoạn phía sau nằm trong đoạn i khi và chỉ khi nó có

$R \leq R_i$. Ta duy trì giá trị nhỏ nhất của R đã gặp (min_R). Nếu $\text{min_R} \leq R_i \implies$ đoạn i chứa đoạn khác!

3. **Đếm số lượng chính xác:** Sử dụng Cây Fenwick (Binary Indexed Tree) trên tọa độ R đã nén để đếm số lượng đoạn thỏa mãn trong $O(N \log N)$.

3.5. Quy hoạch Động trên Đoạn có Trọng số (Weighted Interval Scheduling)

Bài toán: Cho N công việc, công việc i thực hiện trong khoảng $[L_i, R_i]$ và mang lại tiền công W_i . Không thể thực hiện đồng thời hai công việc có thời gian giao nhau. Tìm tổng tiền công lớn nhất có thể thu được.

Thuật toán $O(N \log N)$:

1. Sắp xếp các công việc theo thời điểm kết thúc R_i tăng dần.
2. Gọi $dp[i]$ là tổng tiền công lớn nhất khi chỉ xét tập con trong i công việc đầu tiên.
3. Tại công việc i (khoảng $[L_i, R_i]$, trọng số W_i):

4. **Lựa chọn 1:** Không làm công việc $i \implies dp[i] = dp[i - 1]$.
5. **Lựa chọn 2:** Làm công việc $i \implies$ ta chỉ có thể kết hợp với công việc p kết thúc trước khi công việc i bắt đầu ($R_p \leq L_i$). Để tối ưu, chọn p là công việc lớn nhất có $R_p \leq L_i$:

$$dp[i] = dp[p] + W_i$$

6. Công thức truy hồi:

$$dp[i] = \max(dp[i - 1], dp[p] + W_i)$$

Trong đó chỉ số p được tìm kiếm cực nhanh bằng hàm `upper_bound` trên mảng các thời điểm kết thúc R trong $O(\log N)$.

4. Cấu trúc Dữ liệu Động: Quản lý Tập Đoạn Rời rạc (`Dynamic Interval Set`)

Trong các bài toán xử lý luồng truy vấn (Query Online), ta cần liên tục:

- Thêm một đoạn $[L, R]$ mới.
- Tự động gộp đoạn mới với tất cả các đoạn đang có trong tập hợp nếu chúng giao nhau hoặc tiếp xúc nhau.
- Trả về tổng độ dài hợp các đoạn hiện tại trong $O(1)$.

Kỹ thuật sử dụng `std::set` :

Lưu trữ các đoạn rời rạc trong một `std::set<pair<long long, long long>>`, sắp xếp theo đầu mút trái L .

Khi chèn đoạn $[L, R]$:

1. Dùng `lower_bound` tìm đoạn đầu tiên có đầu mút phải $\geq L - 1$ (tiếp xúc hoặc giao nhau).
2. Lần lượt duyệt và xóa (erase) tất cả các đoạn giao nhau với $[L, R]$, đồng thời mở rộng:

$$L = \min(L, it \rightarrow \text{first}), \quad R = \max(R, it \rightarrow \text{second})$$

3. Chèn đoạn gộp lớn nhất $[L, R]$ vào set.

- **Độ phức tạp:** Mỗi đoạn được chèn vào 1 lần và xóa khỏi set tối đa 1 lần $\implies$ Độ phức tạp khấu hao (Amortized Complexity) là $O(\log N)$ **cho mỗi truy vấn**.

5. Trace Thuật toán Từng bước: Merge Intervals

Xét tập 4 đoạn thẳng:

$$\text{Intervals} = \{[2, 6], [1, 3], [8, 10], [15, 18]\}$$

Bước 1: Sắp xếp theo đầu mút trái L tăng dần

$$\text{Sorted} = \{[1, 3], [2, 6], [8, 10], [15, 18]\}$$

Bước 2: Khởi tạo và quét

- Khởi tạo: $cur_L = 1, cur_R = 3$.
- **Xét đoạn thứ 2:** $[2, 6]$:
Vì $2 \leq cur_R$ ($2 \leq 3$) $\implies$ Giao nhau!
Cập nhật: $cur_R = \max(3, 6) = 6$. Khoảng gộp hiện tại là $[1, 6]$.
- **Xét đoạn thứ 3:** $[8, 10]$:
Vì $8 > cur_R$ ($8 > 6$) $\implies$ Không giao nhau!
Lưu đoạn $[1, 6]$ vào kết quả.
Bắt đầu khoảng mới: $cur_L = 8, cur_R = 10$.
- **Xét đoạn thứ 4:** $[15, 18]$:
Vì $15 > cur_R$ ($15 > 10$) $\implies$ Không giao nhau!
Lưu đoạn $[8, 10]$ vào kết quả.
Bắt đầu khoảng mới: $cur_L = 15, cur_R = 18$.
- **Kết thúc vòng lặp:** Lưu đoạn cuối cùng $[15, 18]$ vào kết quả.

Kết quả cuối cùng:

$$\text{Merged} = \{[1, 6], [8, 10], [15, 18]\}$$

Tổng độ dài hợp: $(6 - 1) + (10 - 8) + (18 - 15) = 5 + 2 + 3 = 10$.

⚠ 6. Cạm bẫy Phòng thi & Kỹ thuật Tối ưu (Traps & Pitfalls)

1. **Tràn số 32-bit (int overflow):**
 2. Các tọa độ điểm L_i, R_i trong các bài toán CP thường lên tới 10^9 .
 3. Khi tính tổng độ dài $\sum(R_i - L_i)$, tổng có thể lên tới $N \times 10^9 \approx 2 \cdot 10^{14}$ nếu $N = 2 \cdot 10^5$.
 4. Bắt buộc phải khai báo biến tổng độ dài bằng kiểu `long long`.
 5. **Sai lầm về thứ tự sự kiện trùng tọa độ trong Dò quét (Sweep-line):**
 6. Nếu đề bài cho các đoạn là **đoạn đóng** $[L, R]$, tại cùng một điểm x , hai đoạn tiếp xúc nhau được tính là chồng lấn $\implies$ Sự kiện `START` (+1) **bắt buộc phải được xử lý trước** sự kiện `END` (-1). Nếu sắp xếp ngược lại, số tầng phủ cực đại sẽ bị tính thiếu 1.
 7. **Phân biệt tính rời rạc của số nguyên khi kiểm tra tiếp xúc:**
 8. Hai đoạn số nguyên $[1, 3]$ và $[4, 6]$ trên tập $\mathbb{Z}$ là hai đoạn kề nhau liên tục. Nếu đề bài cho phép gộp các đoạn kề nhau (khoảng cách ≤ 1), điều kiện gộp là $L_{i+1} \leq cur_R + 1$ chứ không phải $L_{i+1} \leq cur_R$.
 9. **Sắp xếp sai chiều trong bài toán Đoạn chứa nhau:**
 10. Khi tìm đoạn chứa nhau, sắp xếp theo L tăng dần là chưa đủ. Bắt buộc phải thêm tiêu chí phụ: **khi L bằng nhau thì R phải giảm dần**. Nếu không sắp xếp R giảm dần, đoạn ngắn hơn đứng trước đoạn dài hơn sẽ làm thuật toán nhận định sai về tính chất bao hàm.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.